

Design, Build, and Deploy a Full AI-Powered Web Application

Timeframe: Final Phase of the Bootcamp

Project Type: Capstone / Portfolio Project

Required Stack: React.js, Node.js, Express, MongoDB

Project Overview

This capstone project is the final demonstration of your software engineering skills. You are required to **design, build, and deploy a full AI-powered web application** using:

- **React.js** for the frontend
- **Node.js and Express** for the backend
- **MongoDB** for the database

This project must demonstrate **end-to-end development skills**, including planning, design, implementation, integration, testing, deployment, and documentation.

This is not only a full-stack application. It must also be **AI-powered**, meaning the final product itself must include at least **one meaningful AI-driven feature** that provides value to the user.

You are also allowed to use AI tools throughout the development lifecycle to help with planning, coding, debugging, enhancement, and documentation. However, you remain fully responsible for the final solution.

Project Theme

Smart Appointment & Service Booking Platform (AI-Powered)

You will build a **web-based appointment and service booking platform** for a real-world service domain such as:

- medical clinic
- tutoring service
- beauty salon
- fitness coach
- consultant office
- repair service
- legal or professional service provider

You may rename and brand the application as you wish, but the core idea must remain a **service booking platform**.

Core Project Goal

Your goal is to build a modern full-stack web application where users can:

- register and log in
- browse available services
- view service details
- book appointments
- manage bookings
- interact with an **AI-powered feature** inside the application

The application must feel like a real product and should be suitable for inclusion in a professional portfolio.

Required Architecture

You must implement the project using **3-tier architecture**:

1. Presentation Layer

Built with **React.js**

Requirements:

- modern React application
- component-based structure
- responsive design
- clean UI/UX
- proper frontend organization

You may use React-related libraries such as:

- React Router
- Redux / Zustand / Context API
- UI libraries if approved
- chart or form libraries if needed

2. Application Layer

Built with **Node.js** and **Express**

Requirements:

- structured backend application
- routes, controllers, and business logic clearly separated
- RESTful communication between frontend and backend
- validation and error handling
- authentication and authorization support

3. Data Layer

Built with **MongoDB**

Requirements:

- MongoDB must be used as the database
- Mongoose is recommended for schema modeling
- entities should be clearly designed
- data relationships should be meaningful and organized

Required AI-Powered Feature

The final application must include at least **one meaningful AI-powered feature** integrated into the user experience.

Examples include:

- AI-powered service recommendation
- AI-generated appointment summary
- AI assistant that helps users choose the right service
- AI-generated support replies or booking guidance
- AI-powered smart search or suggestion feature
- AI-generated service descriptions or customer notes

Important:

The AI feature must be:

- functional

- visible in the final application
- relevant to the business problem
- integrated into the user workflow

The AI feature should not be just a hidden experiment or a development note. It must be part of the application experience.

Software Development Lifecycle (Mandatory)

You must follow the **Software Development Lifecycle (SDLC)** in your project work.

Your submission must demonstrate the following stages:

1. Planning

You must define:

- project idea
- problem statement
- target users
- scope

2. Requirements

You must include:

- user stories
- functional requirements
- non-functional requirements

3. Design

You must create:

- wireframes
- architecture overview
- database design / schema
- page and component planning

4. Implementation

You must:

- build the frontend

- build the backend
- integrate MongoDB
- connect all layers
- implement the AI feature

5. Testing

You must:

- test main workflows
- fix issues
- validate end-to-end behavior

6. Deployment

You must deploy the complete application publicly.

Required Functional Features

Your application must include the following main capabilities:

Authentication

- user registration
 - user login
 - protected areas or actions
 - logout
-

Service Browsing

Users should be able to:

- browse available services
 - view service details
 - understand service duration, category, and description
-

Booking Management

Users should be able to:

- create bookings
 - choose date and/or time
 - view bookings
 - update or cancel bookings if appropriate
-

Dashboard / User Area

The application should provide a dashboard or user area that displays meaningful information such as:

- user bookings
 - service information
 - summaries or upcoming appointments
-

AI-Powered Interaction

The application must include at least one AI feature that adds value to the experience.

Design and UI Requirements

Your application must have a **modern, clean, and responsive design**.

Expectations:

- professional appearance
- logical layout
- user-friendly navigation
- good spacing and readability
- responsiveness across screen sizes
- polished frontend presentation

This is a capstone project, so visual quality matters in addition to technical quality.

Project Structure Requirements

Your project must follow **separation of concerns** and modern project organization.

Expectations:

- frontend and backend clearly separated
 - clean folder structure
 - components, services, routes, controllers, models, etc. organized properly
 - no monolithic files
 - environment variables handled properly
-

AI Usage Policy

You are **allowed and encouraged** to use AI throughout the project lifecycle.

AI may be used for:

- brainstorming features
- writing user stories
- creating wireframes or content drafts
- generating starter code
- debugging errors
- improving code structure
- refactoring logic
- generating documentation
- enhancing the UI or UX
- helping design the database
- helping implement the AI-powered feature itself

However:

You are still responsible for:

- understanding the code you submit
- validating all AI-generated output
- testing the project thoroughly
- making final technical decisions
- ensuring the application works correctly

AI is a **development assistant**, not a replacement for engineering judgment.

Deployment Requirements

You must deploy the application publicly.

Recommended platforms:

- **Frontend:** Netlify or Vercel
- **Backend:** Render or Heroku
- **Database:** MongoDB Atlas

Other equivalent platforms may be used if approved.

Your application must be:

- publicly accessible
 - functional
 - deployable from your submitted codebase
-

Git and GitHub Requirements

You must use **Git** and upload the full project to a **GitHub repository**.

Mandatory GitHub Requirements:

- upload **all source files**
- use a clean and professional repository structure
- make **at least 15 commits**
- commit messages must be meaningful
- create a **README.md** file

Examples of meaningful commit messages:

- Initialize frontend and backend structure
- Add user model and MongoDB connection
- Implement registration and login routes
- Create booking form and service list
- Add booking dashboard
- Integrate AI recommendation feature
- Improve validation and error handling
- Refactor frontend components
- Fix deployment issues
- Final cleanup and README update

Important:

Do not upload everything in one or two commits.
Your Git history should show a real development process.

README File Requirements

Your GitHub repository must include a **README.md** file.

The README should contain:

- project title
 - project overview
 - tech stack
 - key features
 - AI-powered feature description
 - setup instructions
 - how to run the frontend
 - how to run the backend
 - environment variable notes
 - deployment links
 - short explanation of how AI was used during development
-

Required Deliverables

You must submit:

1. GitHub Repository Link

The repository must include:

- all source files
- at least 15 commits
- README file
- full frontend and backend code

2. Live Deployment Link

The deployed application must be publicly accessible.

3. Documentation File

A Word or PDF document must include:

- project overview
- user stories
- wireframes
- architecture explanation
- MongoDB schema / model explanation

- AI-powered feature explanation
 - summary of AI usage during development
-

Professional Expectations

This capstone should be treated like a real software engineering project.

You are expected to demonstrate:

- planning before coding
- organized architecture
- thoughtful implementation
- testing and debugging
- deployment readiness
- professional presentation
- responsible use of AI

This project should be something you can confidently show to a potential employer.

How the Project Will Be Evaluated (Pass / Fail)

This project will be evaluated using a **Pass / Fail** decision based on the following areas.

1. Full-Stack Functionality

The application must work as a complete full-stack system:

- frontend works
- backend works
- MongoDB integration works
- major user flows work correctly

If core functionality is missing or broken, the project may fail.

2. Required Technology Stack

The project must clearly use:

- React.js
- Node.js

- Express
- MongoDB

If the required stack is not followed, the project may fail.

3. AI-Powered Feature

The final application must include at least one meaningful AI-powered feature.

The feature must:

- be integrated into the app
- add user value
- function correctly
- be clearly explained in the documentation

If the application is full-stack but not truly AI-powered, it may not satisfy the capstone requirement.

4. Architecture and Separation of Concerns

The project should demonstrate:

- frontend/backend separation
- organized backend structure
- organized React structure
- clean project architecture

If the project is overly disorganized or monolithic, it may not pass.

5. SDLC Evidence

The student must show evidence of:

- planning
- user stories
- wireframes
- design thinking
- implementation process

If the project lacks planning artifacts and SDLC evidence, it may not pass.

6. Code Quality and Best Practices

The project should show:

- readable and maintainable code
- meaningful naming
- reasonable structure
- error handling
- professional coding habits

If the code is extremely poor, chaotic, or clearly unreviewed, the project may fail.

7. UI/UX and Responsiveness

The project should have:

- a clean modern design
- reasonable responsiveness
- usable interface
- polished presentation

If the application works technically but is extremely weak in presentation and usability, it may be considered incomplete.

8. Deployment

The application must be deployed publicly and function in its deployed environment.

If no deployment is provided, or deployment is broken, the project may fail.

9. Git and GitHub Usage

The project must:

- be uploaded to GitHub
- include all files

- include at least 15 commits
- include a README file

If GitHub requirements are missing, the project may fail.

10. Responsible Use of AI

The student must show responsible AI usage by:

- explaining where AI was used
- demonstrating understanding of the generated work
- validating and testing the solution

If the project appears to be blindly copied from AI without understanding, it may fail.

Pass Decision

A project should **pass** if it:

- meets the core technical requirements
- uses the required stack
- includes a meaningful AI-powered feature
- is deployed successfully
- shows SDLC evidence
- is submitted through GitHub with at least 15 commits and a README
- demonstrates professionalism and understanding

Fail Decision

A project may **fail** if it:

- is incomplete
 - does not use the required stack
 - is not truly AI-powered
 - is not deployed
 - is missing GitHub requirements
 - lacks planning/documentation
 - shows poor code quality or lack of understanding
-